String Continued :-

30 June, 2024

Substring :-

var str = " this is nitin ";

↳ isko duplicate karo.

console.log (str.substring ( ))

(starting index, no. of characters)

JS mei string immutable hoti hai (matlab string cant be changed)

Slice ⇒ console.log (str.slice (0, 3));

starting index

Slice mei aap -ve value de sakte ho, pichhe se copy karne ke liye.

jis character tak chahiye uska number naaki index.

console.log(str.substring(0));

console.log(str.slice(-6, -3));

indexof
concat
slice
→ They all 3 can be used on both array & string.

str.trim() → extra space aage & piche se remove kar dega

Regex Pattern → special character ko type na kare user usko liye

str.replace

str.replaceAll

arr.join(" ");

Datatype → ① Primitive Datatype

Number

Bool

undefined

String

Null

② Referenced datatype

object

array

function

object ka typeof null kyu hai? kyu?

Stack overflow 18808226

Value and reference ka

→ pass by reference object & its types ke cases mei hoga

→ pass by value primitive datatype ke case mei hoga.

object ke sabhi keys ka datatype hamesha string hota hai but you can store any non string value in it any time.

Object → insertion, creation, deletion, updation

Loops :-

```
for (var i=0 ; i < nums.length; i++)
{
    console.log (nums [i]);
}
```

for of →

```
for (var num of nums){
    console. log (num);
}
```

→ num variable /array ban jayega.

for of → Array ke liye use hota hai

for in → object ke liye use hota hai

```
for (var key in person) {
    console.log(person[key]);
}
```

Eloquent JS youdont know JS — book

for in can be applied in both object & array

for of sirf array par lagti hai

Homework :- ① Reverse a string w/o using reverse method

"ABCD"

"DCBA"

② "SOME STRING"

reverse individual word

"EMOS GNIRTS"

③ Remove special characters "AB#XY@12p"

Q1 Reverse a string :-

↳ pehle ABCD ko split karo → string to array.

['A', 'B', 'C', 'D'] → isko reverse karo, then ['D', 'C', 'B', 'A'] ko join kar de.

Q2

Q3 Remove Special Character :-

```
str = "AB #P4@12"

str.replace(/[^a-zA-Z0-9 ]/g, '');
```

/ → ye use karne se kya hua?

[^a-zA-Z0-9] → space is there

'' → nothing in b/w

a-zA-Z0-9 → inko chhodkar jo bhi character hoga string mei wo replace ho jayega. But

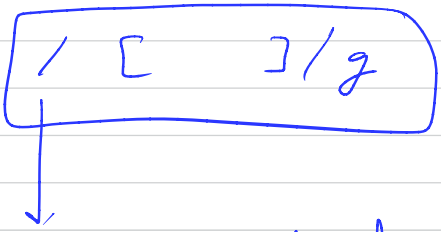
actually mei delete hoga kyuki // ke beech mei space nahi hai.

/ / → deli miter

global ← g → taaki replace sirf first attempt mei nahi balki har character mei ho.

#ABAC# → agar g use nahi hoga then dusra # gayab nahi hoga.

Delimiters ka use flags and aapke expression ko separate karne ke liye use hota hai.



begining & ending of expression batane ke liye → jaise hum double quotes use karte hai.

Samarth Sir

(video - 18)

10 October, 2024

Number :-

↳ 10, 20.5, -5, etc. sab number datatype mei included hai.

```
let a = 'sam';
let b = "d"
```

→ both are strings

Backtick :- Jab apne string expression mei kisi variable ko evaluate karna ho, then we use backtick and ${ }. Check code on the next page.

```
let firstName = 'puneet';
let lastName = 'superstar';

let state = 'delhi';
let favLang = 'eaaahhhhhhhhhh';


let greeting = "kaise ho mere reels ke chahane vaalo aaj kal ke nalle log
khudko " + firstName + lastName + " samajhte hai jo " + state + " mei
rehte hai " + "un sabko mera " +  favLang;


// ---- string template literals --------
let greeting2 = `kaise ho mere reels ke chahane vaalo aaj kal ke nalle log
khudko ${firstName} ${lastName}  samajhte hai jo ${state} mei rehte hai un
sabko mera ${favLang}`;
```

l1

l2

writing l2 is better than l1. Hame "+" use nahi karna pada.

greeting 2
↳ this is string template literal.

In JavaScript, **template literals** (also called template strings) are a way to work with strings that provide more flexibility than traditional string concatenation. They allow embedding variables, expressions, and even multiline strings more easily.

Key Features of Template Literals:

1. **Backticks (`)** are used instead of single or double quotes.
2. **Variable interpolation** using `${}` to embed expressions within the string.
3. **Multiline strings** without needing escape characters for new lines.

Syntax

js Copy code

```
const variable = "world";
const greeting = `Hello, ${variable}!`;
console.log(greeting);  // Output: Hello, world!
```

Numbers :-

```
let num = 10;
console.log ( typeof (num) );
```

↳ o/p → number

typeof ( ) → to find the datatype.

```
let str = "abc";
console.log ( typeof (str) );
```

↳ o/p → string

```
let num = 10;
let num2 = 10.5;
let num3 = -100;
let str = "samarth"

console.log(num);
console.log(num2);
console.log(num3);


console.log(typeof(num));
console.log(typeof(num2));
console.log(typeof(num3));

console.log(typeof(str));
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[Running] node "/Users/nitinkumar/Desktop/WebD
10
10.5
-100
number
number
number
string
string
string

[Done] exited with code=0 in 0.174 seconds
```

Q guess the output :-

```
let str = "samarth"
let str1 = 'vohra';
let ans = str + str1;

let ans1 = str - str1;

let ans2 = str * str1;

let ans3 = str / str1;

console.log(ans);

console.log(ans1);

console.log(ans2);

console.log(ans3);
```

o/p

What is NaN?

Note -, *, / → these mathematical operations won't work with strings

only "+" will work, it will do concatination.

+, -, *, / → they will work with numbers only.

```
let n1 = 100;

let n2 = 30.9;

console.log (n1-n2);
```

↳ o/p → 69.1

yaha C++ ka jaise int, float ka scene nahi hai.

```
let n1 = 100;
let n2 = 1000;
console.log( n1 / n2);
```

↳ o/p → 0.1

% → it is used to get remainder.
↳ it is called modulo.

jaise Maths mei BODMAS follow hota hai, waise hi JS mei PEMDAS follow hota hai.

P → parenthesis
e → exponential
m → multiply
d → divide
a → add
s → subtract

Q
```
let ans = (22-4) * (22/2) +4;
```

iss expression mei sabse pehle kya evaluate hoga?

① Sabse pehle parenthesis chalega

⇓

(22-4) ⇒ 18

② Then again parenthesis chalega

(22/2) ⇒ 11

③ Multiplication hoga :-

18 * 11 ⇒ 198

④ Addition hoga :-

198 + 4 = 202

Q Int ans 2 = (12/4) / (22*2) - 14;

① 3

② 3 / 11

↓

③ 0.26 - 14 ⇒ -13.74

④ -13.74

Q let ans 3 = 22 - 4 * (22/2) / 4;

① (22/2) ⇒ pehle bracket wala evaluate hoga

(11)

② then multiply hoga ⇓

22 - 4 * 11 / 4

⇓

Then division 22 - 44 / 4

⇓

Then subtraction 22 - 11

⇓

11

More About Variables :- (57:00)

let nit = 100; ✓

let nit = "Kumar"; ✗

Redeclaration not possible with let

more about variables > JS app.js > ...

```
1   let nit = 'kumar';
2   let nit = 100;
```

let nit = 100;

nit = "Sharma";

↳ update can be done with let.


```
4   let ab = 'qwerty';
5
6   console.log(typeof(ab));
7
8   ab = true;
9
10  console.log(typeof(ab));
11
12  ab = 100;
13
14  console.log(typeof(ab));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[Running] node "/Users/nitinkumar/Desktop/WebD/Samarth \
string
boolean
number

[Done] exited with code=0 in 0.181 seconds
```

⇒ In this example, we can see datatype is getting changed over and over again.

js mei variables are weakly typed.

TS is a weakly typed programming language.

In last screenshot, we can see that datatype of variable is getting changed.

In C++ it is not possible, C++ is strongly typed programming language.

why js is weakly typed PL?

JavaScript is considered a **weakly typed** programming language because it does not enforce strict type rules, allowing implicit type conversions and flexibility in how variables are used. Here's a breakdown of what that means:

1. No Need to Define Data Types

In JavaScript, you don't have to specify the data type of a variable when declaring it. For example, in strongly typed languages like Java or C++, you'd need to specify whether a variable is an integer, a string, etc., but in JavaScript:

```js
let x = 10;      // 'x' is a number
x = "hello";     // 'x' is now a string
```

You can change the type of a variable without explicitly converting it. This is unlike **strongly typed** languages where the type is rigid.

2. Implicit Type Coercion

JavaScript automatically converts types when it deems necessary. For instance, in the following example:

```js
let result = '5' + 5;   // "55" (string)
```

JavaScript coerces the number `5` into a string because one of the operands is a string. This behavior can sometimes be surprising and can lead to unexpected results.

Similarly:

```js
let result = '5' - 2;   // 3 (number)
```

Here, the string `'5'` is implicitly converted to a number because subtraction is an arithmetic operation.

you can study from gpt ⇐

In JS, the final datatype gets decided on the run time. That's why JS is called dynamically typed PL.

```
4   let ab = 'qwerty';
5
6   console.log(typeof(ab));
7
8   ab = true;
9
10  console.log(typeof(ab));
11
12  ab = 100;
13
14  console.log(typeof(ab));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[Running] node "/Users/nitinkumar/Desktop/WebD/Samarth \
string
boolean
number

[Done] exited with code=0 in 0.181 seconds
```

→ Is code mei ab ka final datatype code run karne par hi pata chalega.

C++ is statically typed. It is opposite of dynamically typed.

→ explained on the next page from gpt.

JavaScript is considered a **dynamically typed** language because the type of a variable is determined at **runtime**, not when the code is written. Here's why and how JavaScript behaves as a dynamically typed language:

1. No Type Declaration at Compile Time

In statically typed languages like Java or C++, you must declare the type of a variable when writing code. For example, in C++:

```cpp
int age = 25;   // 'age' is explicitly declared as an integer
```


However, in JavaScript, you don't specify the type of a variable when declaring it. The type is automatically determined based on the value it holds, and it can change dynamically:

```js
let age = 25;        // 'age' is a number
age = "twenty-five"; // 'age' is now a string
```


This dynamic type assignment allows JavaScript variables to hold different types of values at different times, making the language more flexible but potentially prone to errors if not handled carefully.

2. Type Checking at Runtime

In JavaScript, type checking happens at **runtime**, meaning the type of a variable is evaluated while the program is running. This differs from statically typed languages, where types are checked at compile time, and any type errors are caught before the program runs.

For example:

```js
let value = 42;        // 'value' is a number at this point
value = "forty-two";   // 'value' becomes a string at runtime
```


JavaScript does not throw an error if the type changes, as it allows this kind of flexibility.

Boolean :-

```
let hello = true;
console.log(hello);
```

↳ output ⇒ true

```
let score = 100;
let ans = score == 100;
console.log(ans);
```

↳ o/p : true

let ans = score == 100;

↓

pehle RHS ka expression evaluate hoga, then uski value ans mei assign hogi.

== → comparison operator

= → assignment operator

=== strict equality

== loose equality

yaha sirf value check hogi, naaki datatype

1. `==` (Loose Equality)

- **Performs Type Conversion**: When you use `==`, JavaScript attempts to **convert** the values to the same type before comparing them. This process is called **type coercion**.
- **More lenient**: It allows comparison between different types, sometimes leading to unexpected results.

Example:

```js
console.log(5 == '5');      // true (string '5' is coerced to number 5)
console.log(true == 1);     // true (boolean true is coerced to number 1)
console.log(false == 0);    // true (boolean false is coerced to number 0)
console.log(null == undefined);  // true (special case where null and undefined are
```

When to use `==`:

- It is generally **not recommended** unless you're aware of the type coercion process and want to allow JavaScript to handle type conversions implicitly.
- If you know two values may be of different types but conceptually equivalent (like `'5'` and `5`), `==` might seem convenient, but it can introduce subtle bugs.

2. `===` (Strict Equality)

- **No Type Conversion**: When you use `===`, JavaScript compares both the **value and the type**. It **does not** perform any type coercion, meaning both sides of the comparison must be of the same type for the result to be `true`.
- **More precise**: It prevents the unexpected behavior caused by type coercion.

Example:

```js
console.log(5 === '5');     // false (different types: number vs string)
console.log(true === 1);    // false (boolean vs number)
console.log(false === 0);   // false (boolean vs number)
console.log(null === undefined); // false (different types: null vs undefined)
```

When to use `===`:

- It is the **preferred choice** in most cases, as it avoids the confusion that can arise from implicit type conversions.
- Use it when you want to ensure both the **type and value** are the same.

```
let score = "100";

let ans = score === 100;

console.log(ans);
```

→ strict equality

→ o/p ⇒ false

score ⇒ is string

100 → is number

=== ⇒ yaha datatype bhi check hoga with value

```
let n1 = true;
let n2 = false;

let ans = n1 + n2;

console.log(ans);
```

→ 1 + 0 → 1 ● answer hoga

→ + is a mathematical operator, true will be typecasted to 1 and false to 0.

1 + 0 ⇒ 1

[boolean to int ho gya]

```
let n1 = true;
let n2 = false;

let ans = n1 / n2;

console.log (ans)
console.log (typeof (ans));
```

1 / 0 ⇓

Infinity ⇐ o/p 1

typeof (infinity) ⇓

o/p 2 ⇒ number

```
let n1 = 0 ;
let n2 = false;

let ans = n1 / n2;
```

(0/0) = NaN

```
console.log (ans);
console.log (typeof (ans));
```

→ Number

NaN ↓

Not a number

NaN ka datatype is also Number.

∴ In JS, Numbers are super flexible. It can be integer, float, +ve, -ve value, infinity, -infinity, NaN, etc. In sabhi ka datatype Number hai.

```
let n1 = -1;
let n2 = false;
let ans = n1 / n2;
console.log( ans );
console.log( typeof(ans));
```

-infinity ⇐
Number ⇐

Decision Making :-

we studied if-else :-

```
if (true)
console.log ("abc");
console.log ("def");
```

→ condition flow
→ normal flow

→ ye line true condition ke karan chalegi.

→ ye line normal flow ke karan chalegi, true condition to isse pehle wali line par hi khatam ho gyi.

Q if (true)

→ two line ka space

console.log ("abc");

→ ye print hoga kyunki condition true hai. Bhale hi kitni line ka space ho.

Q if (false)
console.log ("def");

→ kuch print nahi hoga.

Q will this code work?

```
{
    console.log ("main chalega");
}
```

O/p ⇒ main chalega. It will work. we have created a block using braces.

```
if()
{

}
elseif()
  {
  }
elseif()
  {

  }

else{

}
```